

Java HashMap Assignment – Employee Directory

1. Assignment Objective

Create a simple **Employee Directory System** using Java `HashMap`.

The application should store:

- Employee ID as the **key**
- Employee name as the **value**

Example:

```
101 → Amit  
102 → Sneha  
103 → Rahul
```

2. Concepts Covered

This assignment helps students practise the following `HashMap` methods:

Method	Purpose
<code>put()</code>	Add or update an entry
<code>get()</code>	Retrieve a value using a key
<code>putIfAbsent()</code>	Add an entry only when the key is not present
<code>containsKey()</code>	Check whether a key exists
<code>containsValue()</code>	Check whether a value exists
<code>remove()</code>	Delete an entry
<code>replace()</code>	Replace the value of an existing key
<code>size()</code>	Find the number of entries
<code>isEmpty()</code>	Check whether the map is empty
<code>keySet()</code>	Retrieve all keys
<code>values()</code>	Retrieve all values
<code>entrySet()</code>	Retrieve all key-value pairs

Method	Purpose
<code>clear()</code>	Remove all entries

3. Guided Implementation Steps

Step 1: Create the Project

Create a Java project named:

EmployeeHashMapAssignment

Create a class named:

EmployeeDirectory

Step 2: Create a HashMap

Create a `HashMap` where:

- Key type is `Integer`
- Value type is `String`

The employee ID should be stored as the key and employee name as the value.

Step 3: Add Employees

Use the `put()` method to add the following employees:

Employee ID	Employee Name
101	Amit
102	Sneha
103	Rahul
104	Priya
105	Karan

Display the complete employee directory.

Step 4: Retrieve an Employee

Ask the user to enter an employee ID.

Use the `get()` method to display the employee name associated with that ID.

Example:

```
Enter employee ID: 103
Employee Name: Rahul
```

Handle the situation when the employee ID does not exist.

Expected message:

```
Employee not found.
```

Step 5: Check an Employee ID

Use `containsKey()` to check whether employee ID `104` is present.

Expected output:

```
Employee ID 104 exists.
```

Step 6: Check an Employee Name

Use `containsValue()` to check whether an employee named `Sneha` exists.

Expected output:

```
Sneha is present in the employee directory.
```

Step 7: Prevent Duplicate Employee IDs

Try to add employee ID `103` with the name `Rohit` using `putIfAbsent()`.

Since ID `103` is already present, its existing value should not be changed.

Display the employee associated with ID `103` after this operation.

Expected result:

```
103 → Rahul
```

Step 8: Update an Employee Name

Use `replace()` to change the name associated with employee ID `105`:

```
Karan → Karan Mehta
```

Display the updated entry.

Step 9: Remove an Employee

Use `remove()` to delete employee ID `102`.

Display an appropriate message:

```
Employee 102 removed successfully.
```

Display the HashMap after removal.

Step 10: Display All Employee IDs

Use `keySet()` to display all employee IDs.

Example:

```
Employee IDs:  
101  
103  
104  
105
```

The order may vary because `HashMap` does not maintain insertion order.

Step 11: Display All Employee Names

Use `values()` to display all employee names.

Example:

```
Employee Names:  
Amit  
Rahul  
Priya  
Karan Mehta
```

Step 12: Display Key-Value Pairs

Use `entrySet()` to display the employee ID and employee name together.

Expected format:

```
Employee ID: 101, Employee Name: Amit  
Employee ID: 103, Employee Name: Rahul  
Employee ID: 104, Employee Name: Priya  
Employee ID: 105, Employee Name: Karan Mehta
```

Step 13: Display the Number of Employees

Use the `size()` method.

Expected output:

Total Employees: 4

Step 14: Check Whether the Directory Is Empty

Use `isEmpty()` to check whether the HashMap contains any employees.

Expected output:

Employee directory is not empty.

Step 15: Clear the Directory

Use `clear()` to remove all employees.

After clearing, use `isEmpty()` again.

Expected output:

All employee records have been removed.
Employee directory is empty.

4. Menu-Driven Challenge

Extend the assignment by creating the following menu:

1. Add Employee
2. Search Employee
3. Update Employee
4. Remove Employee
5. Display All Employees
6. Check Employee ID
7. Display Employee Count
8. Exit

The menu should continue until the user selects option `8`.

5. Business Rules

1. Employee ID must be greater than zero.
 2. Employee name must not be empty.
 3. Duplicate employee IDs should not be accepted.
 4. Search, update and delete operations must handle missing employee IDs.
 5. Display a meaningful message after every operation.
 6. Employee ID should be used as the unique identifier.
-

6. Sample Execution

```
===== Employee Directory =====
```

1. Add Employee
2. Search Employee
3. Update Employee
4. Remove Employee
5. Display All Employees
6. Check Employee ID
7. Display Employee Count
8. Exit

```
Enter your choice: 1
```

```
Enter Employee ID: 106  
Enter Employee Name: Neha
```

```
Employee added successfully.
```

```
Enter your choice: 2
```

```
Enter Employee ID: 106
```

```
Employee Name: Neha
```

```
Enter your choice: 5
```

```
Employee ID: 101, Employee Name: Amit  
Employee ID: 103, Employee Name: Rahul  
Employee ID: 104, Employee Name: Priya
```

Employee ID: 105, Employee Name: Karan Mehta
Employee ID: 106, Employee Name: Neha

7. Student Questions

1. Why should employee ID be used as the key?
 2. What happens when `put()` is called with an existing key?
 3. What is the difference between `put()` and `putIfAbsent()`?
 4. What does `get()` return when the key is not present?
 5. What is the difference between `keySet()`, `values()` and `entrySet()`?
 6. Does `HashMap` maintain insertion order?
 7. Can a `HashMap` contain duplicate keys?
 8. Can multiple keys have the same value?
 9. Why is `entrySet()` preferred when both keys and values are required?
 10. What happens after calling `clear()`?
-

8. Expected Learning Outcome

After completing the assignment, students should be able to:

- Create and initialise a `HashMap`
- Add, retrieve, update and remove entries
- Prevent duplicate keys
- Search using keys and values
- Iterate through keys, values and entries
- Validate user input
- Build a basic menu-driven application using `HashMap`